Project 1: Getting Started

Foundations & Prompting

AI 402 / AI 502 • Generative AI

Summer 2026 • *May/June Term*

Dr. Zach DeBruine

This document gets you from zero to a deployed AI project by **Sunday at 11:59 PM ET**. Whether you have never opened a code editor or you write software for a living, there is a path here that fits.

Read the first three sections. Then jump to the path that suits you.

What is in this document

1. Welcome and how to read this guide
2. Hour zero: the accounts you need
3. Choose your path (decision tree)
4. Path A — Lovable (no-code, full-stack)
5. Path B — Claude Artifacts (fastest prototype)
6. Path C — VS Code + GitHub Copilot (recommended)
7. Path D — Claude Code (for the brave)
8. GitHub fundamentals (everyone needs this)
9. The AI concepts the rubric is measuring
10. Deployment options compared
11. Your README is your build log
12. Evaluating your project
13. API keys and secrets — a safety briefing
14. When you get stuck
15. A suggested 5-day schedule

1. Welcome and how to read this guide

Project 1 is due as a draft this **Sunday at 11:59 PM ET**. By that deadline you will have built and deployed an AI-powered project of your own design. You will hand in two things: a public URL and a GitHub repository. Nothing else.

That can feel like a lot to learn in five days. It is not. The hard part is the first few hours of account setup and choosing a path. After that you are building.

How to read this

Do not read end-to-end. **Read sections 1–3, then jump to your path.** Come back for sections 8–14 as you need them.

A note on what this guide is and is not. This is a starter kit, not a manual. The tools you are about to use change every week. Specific buttons and menu items will look slightly different by the time you read this. That is fine. The point is to know what *kind* of thing you are looking for; AI itself will help you with the rest.

Use AI as you read this

Open Claude or ChatGPT in another tab while you work through this document. Anything that is not clear — ask. Paste the paragraph in. Ask follow-up questions. This is what the course is about.

2. Hour zero: the accounts you need

Set these up first. All are free unless noted. Use the same email everywhere to keep your life simple.

Required (do all of these)

- **GitHub** — github.com. Pick a username carefully; it will appear publicly on your repository URLs and is hard to change later. Then go to education.github.com/pack and apply for the **Student Developer Pack** using your GVSU email. This gives you **GitHub Copilot Pro for free** along with many other perks. Approval can take a few hours to a couple of days; apply today.
- **Claude** — claude.ai. Free tier is enough to start. You will use this for chatting with the model, building Artifacts, and creating Projects (a kind of workspace with persistent context).
- **ChatGPT** — chat.openai.com. Free tier is enough. You want this for cross-model comparison — the syllabus requires using at least two providers across your three projects.
- **Google AI Studio** — aistudio.google.com. Generous free tier including a free Gemini API key. Excellent for very long context tasks (documents, codebases).
- **Lovable** — lovable.dev. Sign in with your GitHub account. Free tier provides a limited number of builds per day, which is plenty for a first project.

Optional paid upgrades (only if you want them)

You will get a passing grade with the free tier alone. These are listed so you know what your money buys, not because you need them.

- **Claude Pro** ($20/mo) — higher message limits on claude.ai, more Artifacts, longer Projects.
- **Claude Code** — a terminal-based agentic coding tool. Either pay-as-you-go via the Anthropic API (a few dollars to start) or via a Claude subscription.
- **OpenAI API** — $5 minimum credit. Needed only if you want to call OpenAI models from your own code (for example, to use Whisper for speech-to-text).

Avoid predatory tools

If you find a tool charging $50–100 a month for "AI writing" or "AI content," walk away. Look at what tokens actually cost from Anthropic, OpenAI, or Google. If a tool is 50–100x more expensive than the underlying model, you can almost always build it yourself for free.

3. Choose your path

There are four practical paths to a deployed project. You can mix and switch. Pick where to start based on your comfort and your project idea.

Quick decision tree

- "I want a real web app fast and I do not want to touch code."
  → **Path A: Lovable**
- "I want to test an idea in 30 minutes and see something on screen."
  → **Path B: Claude Artifacts**
- "I want some control, I am willing to learn a little, and I want the maximum learning curve in this course."
  → **Path C: VS Code + Copilot** (recommended for most students)
- "I am already comfortable in a terminal and want serious power."
  → **Path D: Claude Code**

You are not locked in

Many students start in Lovable to validate an idea, then move to VS Code + Copilot for more control. Others sketch in Artifacts, then rebuild properly elsewhere. Use whatever tool moves you forward.

4. Path A — Lovable (no-code, full-stack)

Lovable scaffolds a complete web application from a description in plain English. It handles the frontend, the backend, the database, and the hosting. You stay in a chat-style interface and never see code unless you want to.

Step-by-step

1. Go to lovable.dev and sign in with your GitHub account.
2. Click **New project**. On the next screen, describe what you want to build in plain English. Be specific about the user, the problem, and what the app should look like.

3. Lovable will scaffold a complete app and show you a live preview.
4. Iterate by typing into the chat panel: "Add a button that does X." "The login flow is broken — fix it." "Change the color scheme."
5. When something breaks, paste the exact error message back and ask Lovable to fix it. Do not rephrase — the exact error is the signal.
6. Click **Connect GitHub** in the project settings. This creates a repository under your GitHub account and syncs every change. Now your code lives on GitHub automatically.
7. Lovable publishes a public URL automatically (something like `your-project.lovable.app`). You can attach a custom domain if you want, but the default URL is fine for this course.

Tips for getting good results from Lovable

- **Scope down.** Build the smallest version that works, then add features. If you try to build everything in one prompt, you will get a mess.
- **Be the product manager.** Describe the user, the goal, the constraints. Avoid giving implementation details unless something is going wrong.
- **Free tier rationing.** Lovable limits messages per day on the free tier. If you run out, switch to another tool for the rest of the day or wait until tomorrow.
- **LLM features in your Lovable app.** You can tell Lovable "add a chat feature powered by Claude" or "use OpenAI to summarize the uploaded text." It will ask you to paste an API key (which it stores as a secret on its backend). It can also help you write a good system prompt for the LLM your app calls.

5. Path B — Claude Artifacts (fastest prototype)

Artifacts let you ask Claude to build a self-contained app — a single HTML file, a React component, an SVG, a small interactive tool — and Claude generates and previews it in the right side of the chat window. This is the fastest way to see something working.

Step-by-step

1. Open claude.ai and start a new chat.
2. Tell Claude what you want to build. Examples:
   - "Build me a single-page HTML app that takes a paragraph of text and returns a sentiment score and three suggested edits, calling the Anthropic API."
   - "Build me a React artifact that quizzes the user on Spanish vocabulary with spaced repetition."
3. Claude generates the artifact in the right panel. Click around. Use it.
4. Iterate: "The button is the wrong color." "Add a copy-to-clipboard feature." "Persist data between sessions."
5. When you are happy, click **Publish** at the top of the artifact. This creates a public shareable URL.

For more serious work — create a Project

A Claude Project is a persistent workspace. You can upload reference documents (research papers, your domain corpus, a style guide) once, and every chat inside the Project has access to them. This is your easy on-ramp to *grounding* (see §9). Set the "Project instructions" field — this is your system prompt.

Getting Artifact code onto GitHub

The artifact's code is visible in the chat. You will need to copy it into a file in a GitHub repository to satisfy the GitHub requirement of the rubric. Either:

- Copy the code, paste it into a file in VS Code, push to GitHub (see §6, §8).
- Or ask Claude: "Write me the exact files I need to put this into a GitHub repository, including a README." Copy each file.

6. Path C — VS Code + GitHub Copilot (recommended)

This path gives you the best balance of control, learning, and transferable skills. VS Code is a free code editor. GitHub Copilot is an AI pair programmer that lives inside it. Together they let you talk your way through building real software — without needing to know much code yourself.

Initial setup

1. Download VS Code from code.visualstudio.com and install it.
2. Open VS Code. Click the **Extensions** icon in the left sidebar (or press `Ctrl+Shift+X`). Search for and install:
   - **GitHub Copilot**
   - **GitHub Copilot Chat**
   - (optional) **Claude Code** (if you want to also use Claude Code inside VS Code)
3. Click the account icon in the lower-left of VS Code and sign in to GitHub. Once your Student Developer Pack is approved, Copilot will activate.
4. Open the Chat panel with `Ctrl+Shift+I` (`Cmd+Shift+I` on Mac). This is where you will spend most of your time.

What the Chat panel can do

- **Model selector** (top of the panel). Choose **Claude Sonnet 4.6** for most work. Drop to **Claude Haiku 4.5** for cheap quick edits. Use **Claude Opus 4.7** only for hard problems or final polish — it is much more expensive in tokens.
- **Approval / Autopilot toggle.** "Ask" mode asks permission before each file edit or terminal command. "Agent" mode lets it run with much less friction. Start in Ask mode until you trust it.
- **Voice input.** Press the microphone icon and talk. Most people communicate intent much faster by voice than by typing.
- **Attachments.** Drag in files, screenshots, or terminal output.

Starting your first project

1. In VS Code: **File** → **Open Folder**. Make a new folder, for example `ai-project-1`, and open it.
2. Open the Copilot Chat panel. Paste in something like:

```
Set up this folder as a new project. I want to build [one-sentence
description of what you want]. Initialize git, create a README.md, create
a .gitignore appropriate for the tech stack you'd recommend, and create
a new GitHub repo under my account called "ai-project-1". Then describe
to me the architecture you suggest before writing any code.
```

3. Read the proposed architecture. Push back if something does not match your goals. Once you agree, say "go ahead."
4. Test as you build. Copilot can start a development server and tell you the URL to open in your browser.

Essential shortcuts

- `Ctrl+Shift+I` — open Copilot Chat
- `Ctrl+I` — inline edit (highlight code in a file, ask for changes in place)
- `Tab` — accept Copilot autocomplete suggestion
- `Esc` — reject a suggestion

7. Path D — Claude Code (for the brave)

Claude Code is a command-line agentic coding tool. It is the most powerful and the most agentic of the four paths. It expects you to be comfortable opening a terminal. If that sentence made you nervous, start with Path C.

1. Install Node.js (the LTS version) from nodejs.org.
2. Open a terminal. Run:

```
npm install -g @anthropic-ai/claude-code
```

3. Navigate to your project folder and run `claude`.
4. Talk to it. It will plan, write files, run tests, and iterate on its own.

Pricing: typically a few dollars in API credits per project, or use a Claude subscription tier that includes Claude Code.

8. GitHub fundamentals (everyone needs this)

Your final submission requires a GitHub repository. Even if you build in Lovable or Claude Artifacts, your code must end up on GitHub. Here is the minimum you need to know.

The mental model

A **repository** ("repo") is a folder on your computer that is tracked by **git** and mirrored to a remote copy on **GitHub.com**. You edit files locally. You **commit** a set of changes with a short message describing what you did. You **push** your commits up to GitHub so they appear on the public page. That is the loop.

The easiest way — through VS Code

1. Click the **Source Control** icon in the left sidebar of VS Code (it looks like a branching tree).
2. If your folder is not a repository yet, click **Initialize Repository**, then **Publish to GitHub**. Choose "public." VS Code creates the GitHub repo for you and pushes your files.
3. After every meaningful change: type a short commit message in the Source Control panel (for example, "add system prompt for summarizer"), click the **checkmark** to commit, then click **Sync** to push to GitHub.
4. Visit `github.com/yourusername/your-repo` to see your project online.

What to commit and what not to commit

Commit:

- All your source files (`.py` , `.js` , `.html` , etc.)
- Your `README.md`
- A `.gitignore` appropriate to your project

Never commit:

- API keys, passwords, or any file containing secrets
- `.env` files
- Generated folders: `node_modules/` , `__pycache__/` , `venv/` , `.next/`

If you accidentally commit a secret

Revoke the key immediately in the provider's console (Anthropic, OpenAI, etc.). Once a key is in your GitHub history, treat it as burned — even if you delete the commit, it may have been scraped. Generate a new key.

9. The AI concepts the rubric is measuring

Five rubric items refer to AI-specific concepts. Here is what each one means in plain language.

System prompt vs user prompt

When you chat with an AI, what you type is the **user prompt**. The **system prompt** is the persistent instruction set above your messages that tells the model who it is, how to respond, what scope it operates in, and what to refuse. The user never sees the system prompt. A good system prompt makes the same model behave reliably across hundreds of different user prompts.

Where to set system prompts in each tool:

- **Claude API / Claude Code** — explicit `system` parameter in the API call.
- **OpenAI API** — a message with `role: "system"` as the first message.
- **Claude.ai web** — create a **Project**, then fill in the "Project instructions" field. That field is your system prompt for every chat in that project.
- **ChatGPT web** — create a Custom GPT, or use the "Custom instructions" setting under your profile.
- **Lovable** — inside your Lovable app, the LLM you call has its own system prompt. Ask Lovable to "write a strong system prompt for the model that powers feature X."
- **GitHub Copilot** — create a file `.github/copilot-instructions.md` in your repo. Copilot reads it on every chat in that workspace.

Prompt engineering techniques

You need at least a couple of these visible in your project:

- **Few-shot examples** — show the model two or three examples of input and the desired output, then give it the real input.
- **Structured output** — tell the model exactly what format to return (JSON with these fields, a markdown table, a numbered list). Then your code can parse it reliably.
- **Chain of thought** — ask the model to think step by step before giving its final answer. Often dramatically improves reasoning.
- **Role and constraints** — "You are a careful editor. You must preserve every numeric value. You may not change quoted text."
- **Self-critique** — ask the model to draft, then critique its own draft, then revise. Useful for higher-stakes outputs.

Grounding

Grounding means giving the model the right context to do its job, instead of hoping it knows. Approaches, easy to hard:

- **Stuff the prompt.** For small reference material, paste it in. "Here is the document. Answer the question only from this document."
- **File attachments.** Many tools let users upload files that go into the model's context.
- **Claude Project knowledge.** Upload documents once, query across them in any chat.
- **Retrieval-Augmented Generation (RAG).** For a larger corpus: split documents into chunks, embed them, store the embeddings, retrieve the most relevant chunks at query time, and inject those chunks into the prompt. Project 1 does not require full RAG — file upload or Project knowledge is enough for grounding credit.

Iteration

The first prompt you write is never the prompt you ship. Iteration means: write a draft, run it on a few test inputs, find where it breaks, change the prompt, run again. Save the versions. Your build log should show this loop.

Evaluation

Before you build, write down what "good" looks like for your project. This does not have to be fancy. Examples:

- "Given these 10 test inputs, the system produces a useful answer in at least 8 of them, judged by me on a written rubric."
- "On a labeled test set of 30 examples, the classifier reaches at least 80% precision on the positive class."
- "Across three model providers on the same input, I can defend why one of them is the right choice."

Then test against that criterion before you ship. Document what happened.

10. Deployment options compared

You must hand in a live URL the instructor can interact with. Here are your practical options.

- **Lovable hosting** — automatic. Comes free with any Lovable project. Good for: anything built in Lovable.
- **Vercel** (vercel.com) — sign in with GitHub, click **Import Project**, pick your repo, click deploy. Auto-redeploys on every push. Free for hobby use. Good for: Next.js, React, static sites, small Node backends.
- **GitHub Pages** — free, hosts purely static sites (HTML/CSS/JS only, no backend). In your repo, **Settings** → **Pages**, then choose a branch. Good for: static portfolios, single-page tools that call APIs from the browser.
- **Netlify** (netlify.com) — like Vercel. Good for: same use cases. Pick whichever you like.
- **Hugging Face Spaces** (huggingface.co/spaces) — free CPU hosting for Gradio or Streamlit apps. Good for: Python data or ML demos.
- **Render** or **Railway** — small free tiers for full backend services. Good for: Python or Node servers, databases.
- **Claude Artifacts publish** — click Publish on any Artifact. Good for: a fully working demo with no backend complexity.

For most students: Lovable's built-in deploy, or Vercel pointing at your GitHub repo.

11. Your README is your build log

The rubric requires a build log. The easiest place to put it is in the `README.md` at the root of your GitHub repo. GitHub renders Markdown automatically on the repo page — this is what visitors see.

What to include

- Project name and a one-sentence pitch.
- The problem you are solving and the audience you are solving it for.
- A link to the live demo.
- A screenshot or animated GIF (drop the file into the repo, link to it in the README).
- How to use it.
- Your key prompts. Paste the system prompt verbatim in a code block.
- Which prompting techniques you used and why.
- Your evaluation: what "good" meant, how you tested, what you found.
- Honest limits: where the system breaks, what it cannot do.
- Which AI tools you used to build it.

Markdown cheat sheet

````
# Big heading
## Medium heading
### Small heading

Regular paragraph text. Leave a blank line between paragraphs.

**bold text** and *italic text*

- bullet item
- another bullet

1. numbered
2. list

[link text](https://example.com)

![image alt text](path/or/url.png)

`inline code`

```python
# code block - triple backticks
def hello():
    print("hi")
```
````

```
> quoted block
```

12. Evaluating your project

You do not need to write a research paper. You do need to show evidence you actually tested your project.

A simple recipe

1. Before you build, write down: *What does a successful run of my project look like?* One sentence.
2. Make a small test set — 5 to 20 examples is plenty. Inputs your system will actually see.
3. Run your system on each one. Record the result.
4. Score each result. Pick the simplest scoring approach that fits:
   - **Quantitative:** accuracy, precision/recall, BLEU, cost per query, response time — whatever number matters for your task.
   - **Qualitative:** score each output on a 1–5 scale against your own rubric. Note where it failed.
   - **Comparative:** run two different models on the same inputs. Which one is better? Why?
5. Put the results in your README. Be honest about the failures.

Two graders worth knowing

- **LLM-as-judge.** For subjective outputs, give a separate LLM the input, your system's output, and a rubric. Ask it to score. Cheap, fast, surprisingly useful for ranking variants.
- **Yourself.** For 5–20 examples, you reading the outputs carefully is often the best signal you can get.

13. API keys and secrets — a safety briefing

If your project calls an AI API directly — OpenAI, Anthropic, Google — you will have an API key. Treat it like a credit card number.

The standard workflow

1. Get your key from the provider's console:
   - Anthropic: console.anthropic.com
   - OpenAI: platform.openai.com/api-keys
   - Google: aistudio.google.com
2. Make a file called `.env` in your project root:

```
ANTHROPIC_API_KEY=sk-ant-...
OPENAI_API_KEY=sk-...
GOOGLE_API_KEY=AI...
```

3. Add `.env` to your `.gitignore` so it is never committed:

```
.env
.env.local
node_modules/
__pycache__/
venv/
```

4. In your code, load the key from the environment, not from a literal string:

```
# Python
import os
key = os.environ["ANTHROPIC_API_KEY"]

// Node
const key = process.env.ANTHROPIC_API_KEY;
```

5. For your deployed app on Vercel/Lovable/Netlify — set the same environment variables in the platform's dashboard. Never put real keys in code that is pushed to GitHub.

The one rule

Do not commit secrets to GitHub. Ever. If you do, revoke the key and create a new one — do not assume deleting the commit is enough. GitHub is scanned constantly by bots looking for leaked keys; an exposed Anthropic or OpenAI key can rack up serious charges within minutes.

14. When you get stuck

You will get stuck. Everyone does. Here is the order in which to ask for help.

1. **Ask AI first.** Open a fresh chat. Paste your project context — what you are building, what you have tried, the exact error message. Treat the AI as a knowledgeable colleague who knows nothing about your project until you tell it. Quick one-line questions get quick useless answers.
2. **Try a different model.** If Claude is stuck on something, try GPT or Gemini. Different models have different blind spots. This is also good practice for the rubric's multi-model requirement.
3. **Search the discussion board.** Someone else may have hit the same wall and posted about it.
4. **Post on the Q&A discussion thread.** Describe the problem, what you tried, where you are stuck. I review this every Monday and address common questions in the Wednesday FAQ.
5. **Book office hours.** Calendly link: calendly.com/zachdebruine. Zoom or in person — DCIH 530L. Use these. This is what they are for.
6. **Email.** debruinz@gvsu.edu. I ask one favor: ask AI first. If your email is something an AI could have answered in 30 seconds, I will probably just send you the AI chat I had.

15. A suggested 5-day schedule

You have today through Sunday. Here is one way to spend that time. Adjust to your life — this is a suggestion, not a contract.

Today (Tuesday) — 2 hours

- Finish account setup (§2).
- Pick your path (§3).
- Sketch your project in three sentences: *What problem? Who is the user? What does a successful run look like?*
- Post your project idea on the Q&A discussion thread so I can react if you are heading somewhere infeasible.

Wednesday — 2 to 3 hours

- Build a minimum viable version. Does not need to be good. Just needs to run. Get the boring scaffolding out of the way: the page loads, the API call works, something happens when you click the button.

Thursday — 2 to 3 hours

- Add the AI engineering. Write your system prompt deliberately. Apply two prompting techniques. Add grounding (file upload, Project knowledge, or pasted context).
- Push everything to GitHub. Commit messages should describe what changed.

Friday — 1 to 2 hours

- Deploy. Get the public URL working end to end. Send it to a friend. Have them try to break it. Fix what they break.

Saturday — 2 hours

- Evaluate. Run your test set. Score the results.
- Write your README / build log. Paste the system prompt. Describe the techniques. Document the evaluation.

Sunday — 1 hour

- One final round of polish.
- Submit by 11:59 PM ET: paste your deployed URL and your GitHub repo URL on Blackboard. Nothing else.

One last thing

Do not aim for a polished product. Aim for a working version on a viable path. The best projects in this course will be the ones that surprise me. Build something you genuinely want to exist. Have fun with it.

— See you on the discussion board.